Theoretical Mathematics Framework

2.1

When evaluating a dance performance, an audience perceives a fluid, continuous evolution of movement across the stage. However, from a structural and mathematical perspective, a formation change can be modeled as a discrete transition from an initial state to a target state. In a performance like the 'FAMOUS' routine, a group of n dancers occupying a set of initial coordinates must transition into a new arrangement defined by n distinct geometric slots on the stage. The underlying rule of a standard formation transition dictates that every dancer must be allocated to exactly one slot, and no two dancers can occupy the same slot simultaneously.

The fundamental problem lies in determining the mapping of individual dancers to these new positions. This problem falls under the branch of discrete mathematics known as combinatorial optimization—the process of finding the most efficient solution from a finite set of possible arrangements.

To understand why advanced optimization is necessary, we must consider the mathematical scale of a formation change. If a choreography involves a group of n dancers, the first dancer can be assigned to any of the n new positions. The second dancer can then take any of the remaining $n-1$ positions, the third can take $n-2$, and so on. Therefore, the total number of unique ways to assign the dancers to the new formation is given by the factorial function:

$$\text{Total Permutations} = n!$$

For a relatively small group of 3 dancers, there are $3! = 9$ possible variations, a number that could technically be evaluated manually. However, with 5 members (and later on an extra 8 back up dancers), there are $5! = 120$ permutations.

When choreographing or utilizing traditional staging software like FORMI Studio, blocking is typically done using human intuition or basic drag-and-drop mechanics. A choreographer cannot mentally compute or compare 120 distinct permutations to find the absolute mathematically optimal paths for a 5-person group. Intuition naturally defaults to a localized, step-by-step arrangement. This highlights the necessity of an algorithmic approach: human creativity excels at designing the visual aesthetic of a formation, but discrete mathematics is required to objectively ensure the transition itself demands the absolute least amount of physical exertion from the group.

- The reason why I chose the Hungarian algorithm

The overarching purpose of the Hungarian algorithm is to solve the Linear Assignment Problem by transforming a complex geometric distance problem into an algebraic matrix filled with strategic zeros.

The algorithm is built on a foundational mathematical theorem: if a constant number is added to or subtracted from all the elements of any row or column in a cost matrix, the optimal assignment for that matrix remains entirely unchanged. Using this property, the algorithm systematically subtracts the minimum values from rows and columns (a process called matrix reduction). By doing this, it strips away the raw distance values and isolates the relative

differences between options. The zeros that emerge in the matrix represent paths of "zero relative cost."

The goal of the algorithm is to manipulate the matrix until it can find a complete set of independent zeros, meaning a selection of zeros where no two share the same row or column. Once an independent zero is assigned to every single dancer, the assignment problem is solved. The algorithm has successfully identified a globally balanced transition layout.

Why not other methods

The brute-force method calculates the total distance traveled for all $n!$ combinations and selects the smallest one. While mathematically foolproof, its algorithmic complexity is $O(n!)$. As demonstrated by the combinatorial explosion above when getting to groups with larger number of members, attempting to run an exhaustive search on a computer for large numbers becomes highly inefficient and mathematically "un-rigorous". The Hungarian algorithm operates at a polynomial time complexity of $O(n^3)$, meaning that as the group size scales, the computation time grows predictably and negligibly, rendering it a highly scalable model for any choreography size.

A more common alternative is a greedy algorithm. In a spatial context, a greedy algorithm would select Dancer 1 and assign them to the absolute closest available target position. It would then take Dancer 2 and assign them to the closest remaining position, repeating this process until all dancers are placed.

While computationally fast ($O(n^2)$), a greedy algorithm focuses entirely on local optimization (making the best choice at the immediate moment) rather than global optimization (making the best choice for the system as a whole).

$$\text{Greedy Selection: } \min(c_{1j}) \rightarrow \min(c_{2j}) \rightarrow \cdots \rightarrow \text{Forced Last Choice}$$

This introduces a massive choreographic flaw: if the first four dancers greedily take all the central, easily accessible spots, the final two dancers might be forced to sprint all the way across the entire length of the stage to fill the remaining outer wings. The Hungarian algorithm avoids this trap completely. By systematically reducing the entire cost matrix simultaneously, it ensures that individual dancers may sacrifice a slightly shorter path so that the sum total of the group's collective movement is minimized.

Explaining the Complexity

The Brute-Force method relies on exhaustive enumeration. To find the optimal assignment, it must calculate the total distance for every single possible permutation of dancers to slots and then compare them.

As established by the fundamental counting principle, if we have n dancers and n slots:

- The first dancer has n possible slots to choose from.
- The second dancer has $(n-1)$ remaining slots.
- The third dancer has $(n-2)$ remaining slots, continuing down to the final dancer who has exactly 1 slot left.

This yields a total number of permutations equal to:

$$P(n) = n \times (n-1) \times (n-2) \times \cdots \times 1 = n!$$

For each individual permutation, the system must perform n addition operations to sum up the total distance traveled by all n dancers. Therefore, the total number of operations required is exactly $n \times n!$.

In Big-O asymptotic notation, we drop lower-order terms and constants to focus on the dominant growth rate. As n approaches infinity, the growth of the factorial term completely dwarfs the multiplier n, establishing the computational complexity as:
$\lim_{n\to\infty} (n \times n!) \propto O(n!)$

This $O(n!)$ complexity represents factorial growth, which is one of the least efficient runtimes in computer science and completely unscalable for choreographic optimization with bigger groups like 20-30 people.

A greedy algorithm makes the locally optimal choice at each stage. To map out how it reaches $O(n^2)$ complexity, we have to count the number of comparisons it makes as it pairs dancers to slots step-by-step.

Imagine we have a distance matrix of n dancers (rows) and n slots (columns).

- Step 1: To find the absolute closest slot for the first dancer, the algorithm must scan through all n slots in their row to find the minimum value. This requires n comparisons.
- Step 2: For the second dancer, one slot has already been taken. The algorithm must look through the remaining $(n-1)$ slots.
- Step 3: For the third dancer, it searches through the remaining $(n-2)$ slots.

This sequential search continues until the last dancer is forced into the final remaining slot (1 comparison). The total number of comparison operations is the sum of an arithmetic series:

$$\text{Total Operations} = n + (n-1) + (n-2) + \cdots + 1$$

Using the algebraic formula for the sum of the first n integers:

$$S_n = \frac{n(n+1)}{2} = \frac{n^2+n}{2} = \frac{1}{2}n^2 + \frac{1}{2}n$$

When applying Big-O notation, we eliminate the constant coefficient ($\frac{1}{2}$) and drop the lower-order linear term ($\frac{1}{2}n$) because the n^2 term dominates the growth rate as n scales. Thus, the greedy approach is classified as:

$$\text{Complexity} = O(n^2)$$

While $O(n^2)$ (polynomial quadratic growth) is computationally incredibly fast and efficient, its mathematical framework is fundamentally unsuited for choreography. Because the greedy algorithm acts as a sequence of independent, isolated steps, it lacks a global perspective.

The Hungarian algorithm achieves global optimization without checking every permutation by transforming the cost matrix through a sequence of algebraic steps (row/column reductions and matrix alterations).

The algorithm's cubic complexity ($O(n^3)$) is a direct result of how it handles the matrix:

- Matrix reductions ($O(n^2)$): Finding the minimum value of each row requires scanning n elements, done across n rows, resulting in $n \times n = n^2$ operations. The same applies to the columns. Thus, the initial setup takes quadratic time.
- The augmenting path loop ($O(n^3)$): The core of the algorithm relies on finding "augmenting paths" to link independent zeros. If a complete set of zeros cannot be found, the algorithm must shift the matrix values.

In the worst-case scenario, finding a single path requires searching through an entire grid of up to n rows and n columns, which takes $O(n^2)$ steps. Because there are exactly n dancers to assign, this path-finding search must be successfully repeated exactly n times. Multiplying the n required assignments by the $O(n^2)$ work required to find each assignment gives us the total maximum computational work:

$$\text{Total Operations} \propto n \times O(n^2) = O(n^3)$$

This classifies the Hungarian algorithm as having cubic polynomial time complexity. For a large 13-member group, instead of dealing with over 6 billion combinations in a brute-force search, the Hungarian algorithm can find the absolute global minimum in roughly $13^3 = 2197$ operational steps. This balance makes it the ideal mathematical framework for my project: it is fast enough to run instantly on standard laptop hardware, yet rigorous enough to guarantee a globally optimized, mathematically proven stage transition.

2.2 Bipartite Graph

A formation change can be formalized as a weighted bipartite graph.

A graph G is formally defined as an ordered pair $G = (V, E)$, where V represents a set of vertices (or nodes) and E represents a set of edges (the lines connecting the vertices).
For a choreography transition, the vertex set V must be partitioned into two distinct, independent subsets such that $V = X \cup Y$. The use of the union symbol ($\cup$) establishes that the total spatial universe of the transition is strictly comprised of the initial positions and the target destinations. No other points on the stage matter except for the ones where the dancers start (X) and where they end (Y).
This structure forms a bipartite graph, which satisfies the condition:

$$X \cap Y = \emptyset$$

This empty intersection mathematically states that the two sets are mutually exclusive. It means a single coordinate point cannot simultaneously act as a point of origin and a final destination within the same transition window. This empty intersection prevents errors in the model, ensuring the algorithm only calculates pathways leading from a dancer (X) to a target slot (Y), rather than wrongly pairing dancers to other dancers or slots to other slots.

We define these sets within the context of the 'FAMOUS' routine as follows:

- The Dancer Set (X): $X = \{x_1, x_2, \ldots, x_n\}$, where each vertex represents a unique physical dancer at their current coordinate position before the formation change occurs.
- The Target Slot Set (Y): $Y = \{y_1, y_2, \ldots, y_n\}$, where each vertex represents a fixed geometric location on the stage floor that must be filled to successfully construct the next formation.

The edge set E consists of ordered pairs connecting vertices from set X to set Y. In a bipartite graph, edges can only exist between vertices of different sets ($x_i \in X$ and $y_j \in Y$) - otherwise Dancer 1 would go and sit on Dancer 2's shoulders haha:

$$E \subseteq \{(x_i, y_j) \mid x_i \in X, y_j \in Y\}$$

In our mapping model, every single dancer in Set X is initially connected to every single target position in Set Y. This forms a complete bipartite graph, denoted as $K_{n,n}$.
However, a standard graph is only binary, or it only tracks connections. It answers a simple yes/no question: "Is it possible for this dancer to walk to this spot?" Because any dancer can physically walk to any spot on an open stage, every single connection would just say "Yes." A standard graph treats every single path as completely identical.

Thus, I need a weighted graph. The weight of an edge is the exact physical distance between a dancer's starting coordinate and a target formation slot. By adding weights, my graph starts to include cost:

- A short, easy path of 0.5 meters gets a low weight ($w = 0.5$).
- A longer sprint of 3.0 meters gets a high weight ($w = 3.0$).

Now the algorithm's entire job is to look at the network and find the assignment that results in the lowest possible combined number.

Let the weight of the edge connecting dancer x_i to target slot y_j be denoted as $w(x_i, y_j)$, or simply c_{ij} (representing the spatial "cost" of that assignment).

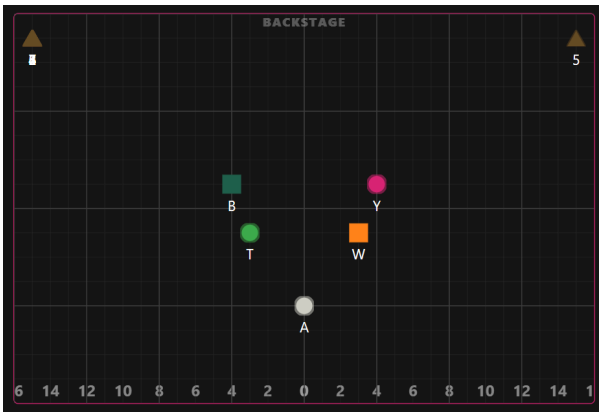
$$\forall x_i \in X \text{ and } \forall y_j \in Y, \quad \exists!\, c_{ij} \in \mathbb{R}^+$$

→ This states that for every dancer-to-slot pairing, there exists a unique, positive real number representing the physical distance between those two points on the stage grid.

To illustrate this abstract notation clearly, we can map out a 3-dancer subset ($n = 3$) of the formation change from the 'FAMOUS' sequence. Let our dancers be $X = \{x_1, x_2, x_3\}$ and our target stage slots be $Y = \{y_1, y_2, y_3\}$.

Every dancer vertex shoots out an edge to all three possible destination vertices, creating a network of $n \times n = 9$ total possible pathways, each carrying its own distinct cost c_{ij}.

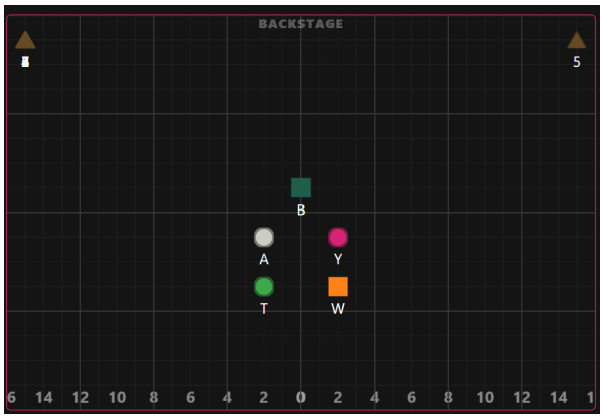




28.8s

Initial position coordinates (Set X at 28.8s)

- Dancer B (x_1): Located at $(-4, 1)$
- Dancer T (x_2): Located at $(-3, -1)$
- Dancer A (x_3): Located at $(0, -4)$
- Dancer W (x_4): Located at $(3, -1)$
- Dancer Y (x_5): Located at $(4, 1)$



35.9s

Target position slots (Set Y at 35.9s)

- Slot 1 (y_1): Located at $(2, -1)$
- Slot 2 (y_2): Located at $(2, -3)$
- Slot 3(y_3): Located at $(0, 1)$
- Slot 4 (y_4): Located at $(-2, -1)$
- Slot 5 (y_5): Located at $(-2, -3)$

With the sets X and Y fully defined, I calculate the absolute physical distance along the straight-line vector connecting every dancer i to every target slot j. The mathematical metric used to evaluate this cost is the Euclidean Distance formula:

$$c_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2},$$

where (x_i, y_i) are the coordinates of a vertex from Set X and (x_j, y_j) are the coordinates of a vertex from Set Y.

Sample calculation with the coordinates $x_1 = (-4, 1)$ and $y_4 = (-2, -1)$

$c_{14} = \sqrt{(-4 - (-2))^2 + (1 - (-1))^2}$

$c_{14} = \sqrt{(-4 + 2)^2 + (1 + 1)^2}$

$c_{14} = \sqrt{(-2)^2 + (2)^2} = \sqrt{4 + 4} = \sqrt{8} \approx 2.83$

	Slot 1	Slot 2	Slot 3	Slot 4	Slot 5
Dancer B	6.32	7.21	4.00	2.83	4.47
Dancer T	5.00	5.39	3.61	1.00	2.24
Dancer A	3.61	2.24	5.00	3.61	2.24
Dancer W	1.00	2.24	3.61	5.00	5.39
Dancer Y	2.83	4.47	4.00	6.32	7.21

Table note: All distances are rounded to 2 d.p. Using the standard Euclidean distance metric.

That's the cost matrix.

Before performing the Hungarian algorithm manually, I will examine the assumptions and limitations of my method first:

Assumptions	Why it is necessary	Impacts
The cost matrix calculates edge weights using the standard Euclidean distance metric, assuming each dancer traverses the absolute shortest geometric path from (x_i, y_i) to (x_j, y_j).	Without assuming linear paths, the distance metric would become variable and non-linear. The algorithm requires static, predictable costs ($c_{ij} \in \mathbb{R}^+$) to initiate row and column reductions.	This slightly distorts choreographic reality. In an actual dance staging, dancers frequently curve their "trajectories" to emphasize specific visual angles or maintain facial visibility toward the audience. Thus, the model may *underestimate* the actual physical distance traveled. However, the straight-line distance is still a very reliable and straightforward benchmark because it provides an objective, reproducible baseline metric for minimum work.
The model assumes that all dancers move at a constant speed and that moving one coordinate unit across any part of the stage costs the exact same amount of physical energy for every individual.	An algorithm cannot intrinsically distinguish if a 3-meter path is being cleared by a powerful or injured dancer. It assumes a homogeneous system where $1 \text{ unit of distance} = 1 \text{ uni}$	In live performance, physical exertion is highly non-linear. Sprinting backward for instance requires significantly more control and energy than walking forward the same distance. While it simplifies human physiology, it ensures mathematical fairness, treating the collective lineup as an idealized, uniform ensemble.

	.		
The bipartite graph treats each dancer and target slot as an infinitely small coordinate dot on a Cartesian plane, completely ignoring the physical volume of the human body.			

Step 1: Reduce the row. Identify the minimum value in each row and subtract it from every single element in that respective row

- Row 1 (Dancer B): Minimum value is 2.83 (at Slot 4)
- Row 2 (Dancer T): Minimum value is 1.00 (at Slot 4)
- Row 3 (Dancer A): Minimum value is 2.24 (at Slot 2 and Slot 5)
- Row 4 (Dancer W): Minimum value is 1.00 (at Slot 1)
- Row 5 (Dancer Y): Minimum value is 2.83 (at Slot 1)

This leaves me with the row-reduced cost matrix C_{row}

3.49	4.38	1.17	0.00	1.64
4.00	4.39	2.61	0.00	1.24
1.37	0.00	2.76	1.37	0.00
0.00	1.24	2.61	4.00	4.39
0.00	1.17	4.00	3.49	4.38

Step 2: Reduce the column: This step ensures that every single destination slot contains at least one "zero-relative-cost" path, making sure that slots are balanced relative to the dancers traveling toward them. Here, only column 3 (Slot 3) needs to be changed since columns 1, 2, 4, and 5 already have a minimum value of 0.00.

3.49	4.38	0.00	0.00	1.64
4.00	4.39	1.44	0.00	1.24

1.37	0.00	1.59	1.37	0.00
0.00	1.24	1.44	4.00	4.39
0.00	1.17	0.00	3.49	4.38

Now to determine if a unique, optimal mapping can be formed using these strategic zeros, I need to draw the absolute minimum number of horizontal/ vertical lines required to cover all the zeros in the matrix. If the number of lines required (k) equals the size of our matrix ($n = 5$), then an optimal assignment is unlocked.
Tracing…

- Line 1 (Horizontal): Row 3 (covers two zeros)
- Line 2 (Horizontal): Row 1 (covers two zeros)
- Line 3 (Vertical): Column 1 (covers two zeros at the bottom left)
- Line 4 (Vertical): Column 4 (covers remaining zeros in column 4)

All zeros are completely covered using only 4 lines. Since $k = 4$ and $n = 5$, the condition $k = n$ is not met, suggesting that the zeros are structurally dependent on one another. If I tried to assign dancers based on this grid right now, two dancers would be forced to battle over the exact same spot. Since a perfect, no-conflict matching does not exist yet, I will …

2.3 Going into actual optimization

Table X:

Keyframe	Dance timestamp	FORMI boundary	Use in our model
K0	49	Formation 10 → 11	Starting formation 11
K1	56.3	Formation 11 → 12	Target 1
K2	58.5	Formation 12 → 13	Target 2
K3	60.6	Formation 13 → 14	Target 3
K4	63.6	Formation 14 → 15	Target 4
K5	65.6	Formation 15 → 16	Target 5

Dancer	Formation 11	Formation 12	Formation 13	Formation 14	Formation 15	Formation 16
B	0, -5	-8, -5	-11, -3	-11, -3	-8, 0	-4, 0
T	0, -2	-10, -4	-2, -1	7, -3	0, -3	1, -3
A	-4, -2	-12, -2	-12, 1	-12, 1	-2, 1	4, 0

W	0, -1	-8, -1	-12, 2	-12, 2	2, -2	2, -2
Y	4, -2	-4, -2	-8, -4	-8, -4	-4, -2	0, -1

1. Baseline: calculate the actual choreography distance

I first need a benchmark representing what the dancers actually do. The dictionary stores each formation as dancer-specific Cartesian coordinates, while `math.dist()` applies the same Euclidean distance metric defined in the mathematical framework. The loop compares each formation only with the immediately following one, because each formation change is treated as an independent assignment problem.

Code block

```
formations = {
    11: {"B": (0,-5),  "T": (0,-2),   "A": (-4,-2),  "W": (0,-1),   "Y": (4,-2)},
    12: {"B": (-8,-5), "T": (-10,-4), "A": (-12,-2), "W": (-8,-1),  "Y": (-4,-2)},
    13: {"B": (-11,-3),"T": (-2,-1),  "A": (-12,1),  "W": (-12,2),  "Y": (-8,-4)},
    14: {"B": (-11,-3),"T": (7,-3),   "A": (-12,1),  "W": (-12,2),  "Y": (-8,-4)},
    15: {"B": (-8,0),  "T": (0,-3),   "A": (-2,1),   "W": (2,-2),   "Y": (-4,-2)},
    16: {"B": (-4,0),  "T": (1,-3),   "A": (4,0),    "W": (2,-2),   "Y": (0,-1)}
}
import math

def distance(p1, p2):
    return math.dist(p1, p2)
for f1, f2 in zip(range(11, 16), range(12, 17)):
    distances = {
        dancer: distance(formations[f1][dancer], formations[f2][dancer])
        for dancer in formations[f1]
    }

    print(f"\nFormation {f1} → {f2}")
    print(distances)
    print("Total:", round(sum(distances.values()), 2))
```

Output:

```
Formation 11 → 12
{'B': 8.0, 'T': 10.198039027185569, 'A': 8.0, 'W': 8.0, 'Y': 8.0}
Total: 42.2

Formation 12 → 13
{'B': 3.605551275463989, 'T': 8.54400374531753, 'A': 3.0, 'W': 5.0, 'Y': 4.47213595499958}
Total: 24.62

Formation 13 → 14
{'B': 0.0, 'T': 9.219544457292887, 'A': 0.0, 'W': 0.0, 'Y': 0.0}
Total: 9.22
```

```
Formation 14 → 15
{'B': 4.242640687119285, 'T': 7.0, 'A': 10.0, 'W': 14.560219778561036, 'Y':
4.47213595499958}
Total: 40.27

Formation 15 → 16
{'B': 4.0, 'T': 1.0, 'A': 6.082762530298219, 'W': 0.0, 'Y': 4.123105625617661}
Total: 15.21
```

Interpretation:
The amount of movement varies substantially between transitions. For example, 13→14 requires only 9.22 units, whereas 11→12 and 14→15 require over 40 units. However, a high actual distance does not automatically mean that the choreography is inefficient.

To determine inefficiency, I need a theoretical minimum to compare against. Therefore, the next step is to run the Hungarian algorithm on the same target positions.

2. Test Hungarian on 11→12 only

Why I started with one transition:
Before automating all five transitions, I tested the procedure on a single case. This makes debugging easier and confirms that the cost matrix and solver behave as intended.

Why the cost matrix is constructed this way:
Each row corresponds to one dancer's starting position and each column corresponds to one possible target slot. Therefore,

cij=d(xi,yj)c_{ij}=d(x_i,y_j)

represents the cost of sending dancer ii to target slot jj. `linear_sum_assignment()` then selects exactly one entry from each row and column while minimizing their total.

Code block [to test Hungarian optimization on Formation 11 → 12 only]

```
from scipy.optimize import linear_sum_assignment
import numpy as np

f1, f2 = 11, 12

dancers = list(formations[f1].keys())
targets = list(formations[f2].values())

cost_matrix = np.array([
    [distance(formations[f1][dancer], target) for target in targets]
    for dancer in dancers
])

rows, cols = linear_sum_assignment(cost_matrix)
```

```
print("\nOPTIMIZED Formation 11 → 12")

for r, c in zip(rows, cols):
    print(dancers[r], "→", targets[c], "distance:", round(cost_matrix[r, c], 2))

print("Optimized total:", round(cost_matrix[rows, cols].sum(), 2))

actual_total = sum(
    distance(formations[f1][d], formations[f2][d])
    for d in dancers
)

optimized_total = cost_matrix[rows, cols].sum()

improvement = (
    (actual_total - optimized_total) / actual_total
) * 100

print("Actual total:", round(actual_total, 2))
print("Improvement:", round(improvement, 2), "%")
```

Output:

```
OPTIMIZED Formation 11 → 12
B → (-8, -5) distance: 8.0
T → (-12, -2) distance: 12.0
A → (-4, -2) distance: 0.0
W → (-8, -1) distance: 8.0
Y → (-10, -4) distance: 14.14
Optimized total: 42.14
Actual total: 42.2
Improvement: 0.13 %
```

What I learned:
11→12 improves only from 42.20 to 42.14 units, or 0.13%. So even though the dancers travel quite far, the original choreography is already extremely close to the minimum-distance assignment. → One transition cannot tell me whether this is typical. Now that the algorithm works, I can apply exactly the same procedure to all five transitions.

3. Run Hungarian across every transition

Why I used a loop:

The mathematical operation is identical for every consecutive pair of formations. Automating it avoids manually rebuilding five different cost matrices and ensures that every transition is evaluated consistently.

Code block [to test all formation changes]

```
for f1, f2 in zip(range(11, 16), range(12, 17)):

    dancers = list(formations[f1].keys())
    targets = list(formations[f2].values())

    cost_matrix = np.array([
        [distance(formations[f1][dancer], target) for target in targets]
        for dancer in dancers
    ])

    rows, cols = linear_sum_assignment(cost_matrix)

    actual_total = sum(
        distance(formations[f1][d], formations[f2][d])
        for d in dancers
    )

    optimized_total = cost_matrix[rows, cols].sum()

    improvement = (
        (actual_total - optimized_total) / actual_total
    ) * 100

    print(f"\nOPTIMIZED Formation {f1} → {f2}")
    print("Actual:", round(actual_total, 2))
    print("Optimized:", round(optimized_total, 2))
    print("Improvement:", round(improvement, 2), "%")
```

Output:

```
OPTIMIZED Formation 11 → 12
Actual: 42.2
Optimized: 42.14
Improvement: 0.13 %

OPTIMIZED Formation 12 → 13
Actual: 24.62
Optimized: 12.65
Improvement: 48.62 %

OPTIMIZED Formation 13 → 14
Actual: 9.22
Optimized: 9.22
Improvement: 0.0 %

OPTIMIZED Formation 14 → 15
Actual: 40.27
Optimized: 34.41
Improvement: 14.57 %

OPTIMIZED Formation 15 → 16
```

Actual: 15.21
Optimized: 15.21
Improvement: 0.0 %

Code block [for more specific info.]

```
def distance(p1, p2):
    return math.dist(p1, p2)
for f1, f2 in zip(range(11, 16), range(12, 17)):

    dancers = list(formations[f1].keys())
    targets = list(formations[f2].values())

    cost_matrix = np.array([
        [distance(formations[f1][dancer], target) for target in targets]
        for dancer in dancers
    ])

    rows, cols = linear_sum_assignment(cost_matrix)

    actual_total = sum(
        distance(formations[f1][d], formations[f2][d])
        for d in dancers
    )

    optimized_total = cost_matrix[rows, cols].sum()
    improvement = ((actual_total - optimized_total) / actual_total) * 100

    print(f"\nOPTIMIZED Formation {f1} → {f2}")

    for r, c in zip(rows, cols):
        print(
            dancers[r], "→", targets[c],
            "distance:", round(cost_matrix[r, c], 2)
        )

    print("Actual:", round(actual_total, 2))
    print("Optimized:", round(optimized_total, 2))
    print("Improvement:", round(improvement, 2), "%")
```

Output:

OPTIMIZED Formation 11 → 12
B → (-8, -5) distance: 8.0
T → (-12, -2) distance: 12.0
A → (-4, -2) distance: 0.0
W → (-8, -1) distance: 8.0
Y → (-10, -4) distance: 14.14
Actual: 42.2
Optimized: 42.14
Improvement: 0.13 %

OPTIMIZED Formation 12 → 13
B → (-8, -4) distance: 1.0
T → (-11, -3) distance: 1.41
A → (-12, 1) distance: 3.0
W → (-12, 2) distance: 5.0
Y → (-2, -1) distance: 2.24
Actual: 24.62
Optimized: 12.65
Improvement: 48.62 %

OPTIMIZED Formation 13 → 14
B → (-11, -3) distance: 0.0
T → (7, -3) distance: 9.22
A → (-12, 1) distance: 0.0
W → (-12, 2) distance: 0.0
Y → (-8, -4) distance: 0.0
Actual: 9.22
Optimized: 9.22
Improvement: 0.0 %

OPTIMIZED Formation 14 → 15
B → (-4, -2) distance: 7.07
T → (2, -2) distance: 5.1
A → (-8, 0) distance: 4.12
W → (-2, 1) distance: 10.05
Y → (0, -3) distance: 8.06
Actual: 40.27
Optimized: 34.41
Improvement: 14.57 %

OPTIMIZED Formation 15 → 16
B → (-4, 0) distance: 4.0
T → (1, -3) distance: 1.0
A → (4, 0) distance: 6.08
W → (2, -2) distance: 0.0
Y → (0, -1) distance: 4.12
Actual: 15.21
Optimized: 15.21
Improvement: 0.0 %

What the results tell me:
The optimization potential is highly uneven:

-11→12: 0.13%

12→13: 48.62%

13→14: 0%

14→15: 14.57%

15→16: 0%

This is important because it shows the algorithm is not simply producing an "improvement" everywhere. Three transitions are already essentially optimal, while 12→13 and 14→15 contain substantial alternative assignments.

Why the next step:
The percentages tell me *where* optimization happens, but not *how*. Therefore, I need to inspect the dancer-to-slot assignments themselves. → Formation 12 - 13 & Formation 14 & 15 have the greatest optimization

4. Print each optimized dancer assignment

Why I added this:

A total distance alone hides the mechanism producing the improvement. Printing every chosen edge allows me to see which dancers are reassigned and how much each optimized path costs.

Code block [Label the target slots by dancer to make the output more readable]

```
Replace: targets = list(formations[f2].values())
With: target_names = list(formations[f2].keys())
targets = list(formations[f2].values())
Then replace the print inside the assignment loop from:
for r, c in zip(rows, cols):
        print(
            dancers[r], "→", targets[c],
            "distance:", round(cost_matrix[r, c], 2)
        )
to:
for r, c in zip(rows, cols):
    print(
        dancers[r], "→", target_names[c] + "'s slot",
        targets[c],
        "distance:", round(cost_matrix[r, c], 2)
    )
```

```
OPTIMIZED Formation 11 → 12
B → B's slot (-8, -5) distance: 8.0
T → A's slot (-12, -2) distance: 12.0
A → Y's slot (-4, -2) distance: 0.0
```

W → W's slot (-8, -1) distance: 8.0
Y → T's slot (-10, -4) distance: 14.14
Actual: 42.2
Optimized: 42.14
Improvement: 0.13 %

OPTIMIZED Formation 12 → 13
B → Y's slot (-8, -4) distance: 1.0
T → B's slot (-11, -3) distance: 1.41
A → A's slot (-12, 1) distance: 3.0
W → W's slot (-12, 2) distance: 5.0
Y → T's slot (-2, -1) distance: 2.24
Actual: 24.62
Optimized: 12.65
Improvement: 48.62 %

OPTIMIZED Formation 13 → 14
B → B's slot (-11, -3) distance: 0.0
T → T's slot (7, -3) distance: 9.22
A → A's slot (-12, 1) distance: 0.0
W → W's slot (-12, 2) distance: 0.0
Y → Y's slot (-8, -4) distance: 0.0
Actual: 9.22
Optimized: 9.22
Improvement: 0.0 %

OPTIMIZED Formation 14 → 15
B → Y's slot (-4, -2) distance: 7.07
T → W's slot (2, -2) distance: 5.1
A → B's slot (-8, 0) distance: 4.12
W → A's slot (-2, 1) distance: 10.05
Y → T's slot (0, -3) distance: 8.06
Actual: 40.27
Optimized: 34.41
Improvement: 14.57 %

OPTIMIZED Formation 15 → 16
B → B's slot (-4, 0) distance: 4.0
T → T's slot (1, -3) distance: 1.0
A → A's slot (4, 0) distance: 6.08
W → W's slot (2, -2) distance: 0.0
Y → Y's slot (0, -1) distance: 4.12
Actual: 15.21
Optimized: 15.21
Improvement: 0.0 %

What I learned:
For 12→13, for example, the large saving is not caused by everyone moving slightly less. It comes from a major reassignment: B→Y,T→B,Y→TB\rightarrow Y,\qquad T\rightarrow B,\qquad Y\rightarrow T while A and W retain their original slots. That reduces total distance from 24.62 to 12.65.

Why the next step: Coordinates such as `(-8,-4)` are difficult to interpret choreographically. I therefore need to attach the original dancer identity to each target slot.

5. Print ACTUAL and OPTIMIZED assignments together

The original choreography provides the control condition:

B\rightarrow B,\quad

T\rightarrow T,\quad

A\rightarrow A,\dots

The optimized assignment provides the mathematical alternative. Showing both makes the changes attributable rather than abstract.

```
ACTUAL Formation 11 → 12
B → B's slot (-8, -5) distance: 8.0
T → T's slot (-10, -4) distance: 10.2
A → A's slot (-12, -2) distance: 8.0
W → W's slot (-8, -1) distance: 8.0
Y → Y's slot (-4, -2) distance: 8.0

OPTIMIZED Formation 11 → 12
B → B's slot (-8, -5) distance: 8.0
T → A's slot (-12, -2) distance: 12.0
A → Y's slot (-4, -2) distance: 0.0
W → W's slot (-8, -1) distance: 8.0
Y → T's slot (-10, -4) distance: 14.14
Actual: 42.2
Optimized: 42.14
Improvement: 0.13 %

ACTUAL Formation 12 → 13
B → B's slot (-11, -3) distance: 3.61
T → T's slot (-2, -1) distance: 8.54
A → A's slot (-12, 1) distance: 3.0
W → W's slot (-12, 2) distance: 5.0
Y → Y's slot (-8, -4) distance: 4.47
```

OPTIMIZED Formation 12 → 13
B → Y's slot (-8, -4) distance: 1.0
T → B's slot (-11, -3) distance: 1.41
A → A's slot (-12, 1) distance: 3.0
W → W's slot (-12, 2) distance: 5.0
Y → T's slot (-2, -1) distance: 2.24
Actual: 24.62
Optimized: 12.65
Improvement: 48.62 %

ACTUAL Formation 14 → 15
B → B's slot (-8, 0) distance: 4.24
T → T's slot (0, -3) distance: 7.0
A → A's slot (-2, 1) distance: 10.0
W → W's slot (2, -2) distance: 14.56
Y → Y's slot (-4, -2) distance: 4.47

OPTIMIZED Formation 14 → 15
B → Y's slot (-4, -2) distance: 7.07
T → W's slot (2, -2) distance: 5.1
A → B's slot (-8, 0) distance: 4.12
W → A's slot (-2, 1) distance: 10.05
Y → T's slot (0, -3) distance: 8.06
Actual: 40.27
Optimized: 34.41
Improvement: 14.57 %

Interpretation for 12→13:
Actual T travels 8.54 units to T's slot, whereas the optimizer sends Y to T's slot for only 2.24 units. Similarly, T can reach B's target in only 1.41 units. These exchanges explain much of the 48.62% saving.

Interpretation for 14→15:
The optimizer changes every member's intended destination, but the saving is only 14.57%. This is already a clue that geometric efficiency may require much more disruption of choreographic roles in some transitions than others.

Next step: Numbers still don't reveal spatial problems such as crossing trajectories or dramatic role swaps. So the next step is visualization.

6. Visualize the actual paths + optimized paths

The numerical model assumes straight-line movement, so plotting those same vectors provides a direct visual representation of the model's assumptions. Each line corresponds to one edge actually used by the choreography: $(x_i^t, y_i^t) \to (x_i^{t+1}, y_i^{t+1})$

Why `plt.axis("equal")` matters:
Without an equal axis scale, one unit horizontally could appear visually different from one unit vertically. That would distort the geometry and make path lengths misleading.

Why visualize 12→13 and 14→15 specifically: They were selected *after* the full quantitative comparison because they have the largest optimization gains. So they are case studies chosen from the results.

Why the next step:
I now need to plot the Hungarian paths using the same coordinates and scale, so actual and optimized movement can be visually compared.

```
f1, f2 = 12, 13

plt.figure()

for dancer in formations[f1]:
    x1, y1 = formations[f1][dancer]
    x2, y2 = formations[f2][dancer]

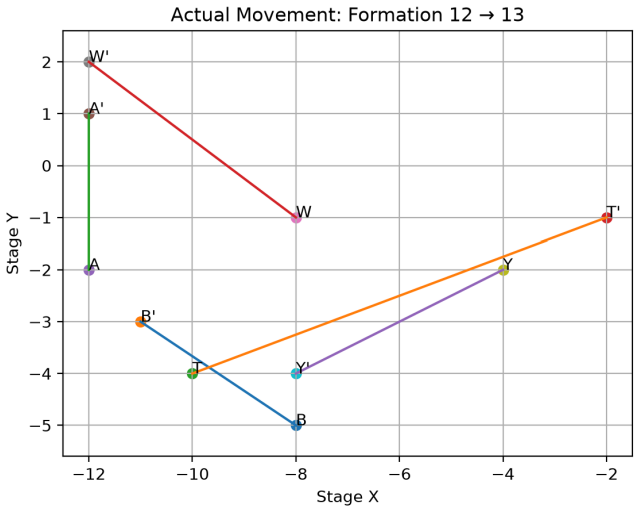
    plt.scatter(x1, y1)
    plt.scatter(x2, y2)

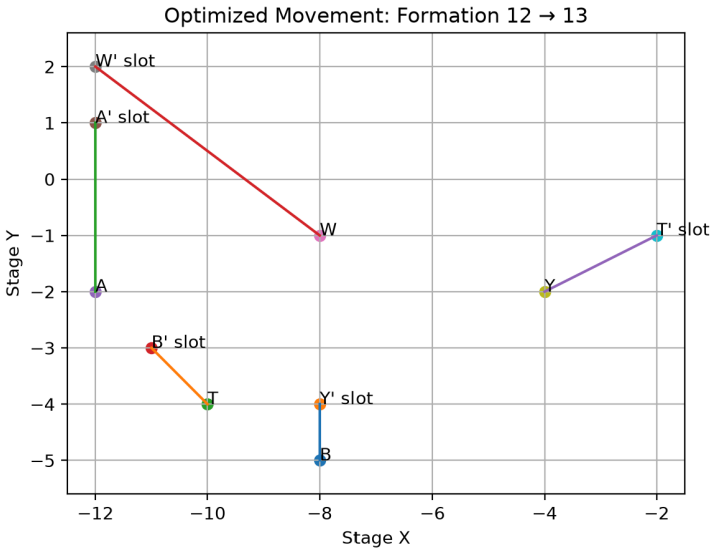
    plt.plot([x1, x2], [y1, y2])

    plt.text(x1, y1, dancer)
    plt.text(x2, y2, dancer + """)

plt.title("Actual Movement: Formation 12 → 13")
plt.xlabel("Stage X")
plt.ylabel("Stage Y")
plt.grid(True)
plt.axis("equal")

plt.show()
```





Formation change 14 → 15

```
f1, f2 = 14, 15
plt.title("Actual Movement: Formation 14 → 15")
```

```
f1, f2 = 14, 15

dancers = list(formations[f1].keys())
target_names = list(formations[f2].keys())
targets = list(formations[f2].values())

cost_matrix = np.array([
    [distance(formations[f1][dancer], target) for target in targets]
    for dancer in dancers
])

rows, cols = linear_sum_assignment(cost_matrix)

plt.figure()

for r, c in zip(rows, cols):
    dancer = dancers[r]

    x1, y1 = formations[f1][dancer]
    x2, y2 = targets[c]

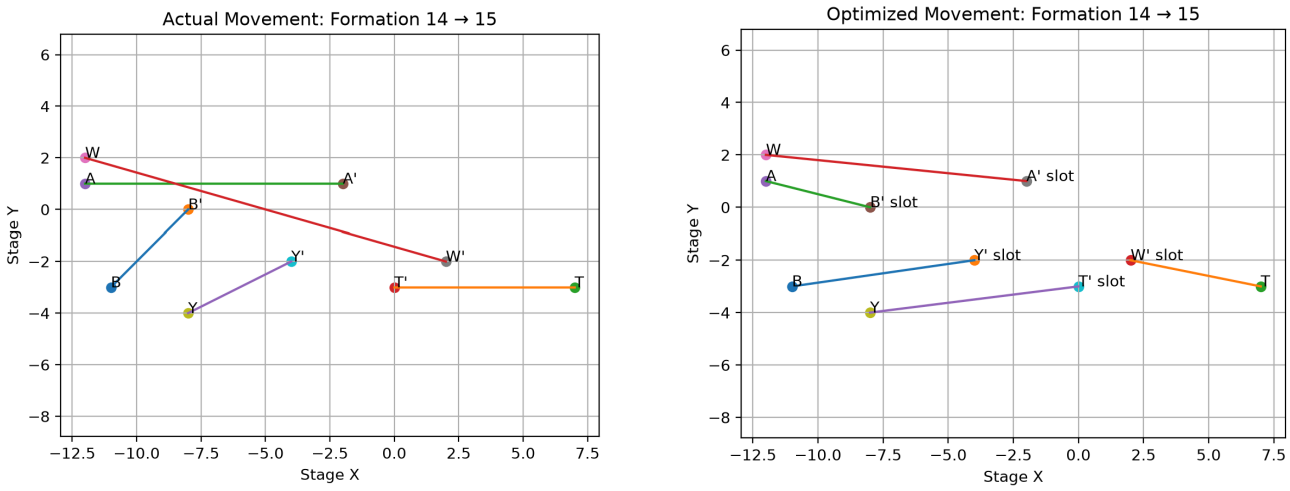
    plt.scatter(x1, y1)
    plt.scatter(x2, y2)

    plt.plot([x1, x2], [y1, y2])

    plt.text(x1, y1, dancer)
    plt.text(x2, y2, target_names[c] + "' slot")
```

```
plt.title("Optimized Movement: Formation 14 → 15")
plt.xlabel("Stage X")
plt.ylabel("Stage Y")
plt.grid(True)
plt.axis("equal")

plt.show()
```



Why the plotting code rebuilds the cost matrix: The graph must use the exact assignment returned by the Hungarian algorithm. Therefore, I reconstruct the same matrix and recover `rows, cols` rather than manually entering the optimized destinations.

What the graphs reveal: For 12→13, the optimized paths are visibly much shorter, matching the 48.62% numerical reduction. But they also demonstrate that achieving this saving requires B, T, and Y to exchange intended roles. For 14→15, the geometric improvement is smaller despite widespread slot reassignment.

Why this tells us what to do next: Up to this point the objective function is: $C_{ij} = D_{ij}$

So the model effectively assumes: "All assignments are equally acceptable artistically; only distance matters." The visualizations show why that assumption is too weak for choreography.

2.4 Adding weighting!

- Center preservation: Arguably the most important thing in K-Pop choreography.

Particular members are often intentionally emphasized because they are delivering the current vocal or rap line, performing a distinctive movement, or acting as the visual focus of that section. Reassigning another dancer to this position may therefore reduce movement distance while weakening the intended distribution of attention within the choreography.

For this reason, the model should distinguish between ordinary target slots and protected focus slots. Assigning a non-intended dancer to a protected slot will receive an additional penalty in the cost matrix, making the Hungarian algorithm less likely to replace the intended focal member purely to reduce travel distance.

For the selected formations:

- Formation 12: T
- Formation 13: T. Although T's target position is not geometrically central, he is the only member performing prominent hand movements while the other members remain comparatively stationary. This makes him the visual focus despite his off-center position.
- Formation 14: T.
- Formation 15: T. W's position is also treated as partially protected because the choreography requires W to remain beside T. Therefore, this formation contains one primary focus constraint (T) and an additional positional relationship involving W.

1. Define the weighting rule

- Purpose: Turn "T should remain the focus" into something the algorithm can understand. For transition 12 → 13, the relevant focus is the target formation (13). So for now we protect T's slot in Formation 13.

My original cost is: $C_{ij} = D_{ij}$. Now if someone other than T tries to take his intended slot: $C'_{ij} = D_{ij} + 10$. This makes the assignment more expensive.

```
FOCUS_PENALTY = 10

def add_focus_penalty(cost_matrix, dancers, target_names, focus_dancer):
    weighted_matrix = cost_matrix.copy()

    for i, dancer in enumerate(dancers):
        for j, target_name in enumerate(target_names):

            if target_name == focus_dancer and dancer != focus_dancer:
                weighted_matrix[i, j] += FOCUS_PENALTY

    return weighted_matrix
```

2. Apply the penalty to 12 → 13 ONLY

- Purpose: Test whether a focus penalty of 10 is strong enough to make T keep T's

intended slot.

Why this way:
We keep two matrices:

- `cost_matrix` = real physical distance
- `weighted_matrix` = distance + artistic penalty

The Hungarian algorithm chooses using the weighted matrix, but we still report the real distance separately. That prevents the penalty from being mistaken for actual movement.

```
# WEIGHTED TEST: Formation 12 → 13

f1, f2 = 12, 13

dancers = list(formations[f1].keys())
target_names = list(formations[f2].keys())
targets = list(formations[f2].values())

cost_matrix = np.array([
    [distance(formations[f1][dancer], target) for target in targets]
    for dancer in dancers
])

weighted_matrix = add_focus_penalty(
    cost_matrix,
    dancers,
    target_names,
    focus_dancer="T"
)

rows, cols = linear_sum_assignment(weighted_matrix)

print("\nWEIGHTED Formation 12 → 13")

for r, c in zip(rows, cols):
    print(
        dancers[r], "→", target_names[c] + "'s slot",
        "distance:", round(cost_matrix[r, c], 2),
        "weighted cost:", round(weighted_matrix[r, c], 2)
    )

print(
    "Actual movement distance:",
    round(cost_matrix[rows, cols].sum(), 2)
)
```

Output:

WEIGHTED Formation 12 → 13
B → Y's slot distance: 1.0 weighted cost: 1.0
T → B's slot distance: 1.41 weighted cost: 1.41
A → A's slot distance: 3.0 weighted cost: 3.0
W → W's slot distance: 5.0 weighted cost: 5.0
Y → T's slot distance: 2.24 weighted cost: 12.24
Actual movement distance: 12.65

Interpretation: A penalty of 10 is not enough. The algorithm still prefers letting Y take T's slot, because even after adding +10, that overall assignment is still cheaper than reorganizing everyone so T keeps it.

3. Test several weights systematically

Replace:

FOCUS_PENALTY = 10

def add_focus_penalty(cost_matrix, dancers, target_names, focus_dancer):

with: def add_focus_penalty(cost_matrix, dancers, target_names, focus_dancer, penalty):

Then replace: weighted_matrix[i, j] += FOCUS_PENALTY

with: weighted_matrix[i, j] += penalty

Why: now the same function can test 0,5,10,15,…0,5,10,15,\dots instead of hard-coding one arbitrary value.

```
for penalty in [10, 15, 20, 25, 30]:
    weighted_matrix = add_focus_penalty(
        cost_matrix,
        dancers,
        target_names,
        focus_dancer="T",
        penalty=penalty
    )
```

```
    rows, cols = linear_sum_assignment(weighted_matrix)

    print(f"\nFOCUS PENALTY = {penalty}")

    for r, c in zip(rows, cols):

        print(dancers[r], "→", target_names[c] + "'s slot")

    print(

        "Movement distance:",

        round(cost_matrix[rows, cols].sum(), 2)

    )
```

```
FOCUS PENALTY = 12
B → Y's slot
T → T's slot
A → B's slot
W → A's slot
Y → W's slot
Movement distance: 24.37
This is the first one where T → B's slot change to T → T's slot
Pure Hungarian=12.65
Focus-preserving =24.37
Actual=24.62
→ So once T's focus is preserved, the improvement falls from 48.62% → about 1.02%.
```

Interpretation: A focus penalty of 12 was the minimum tested weight required to preserve T's intended focal slot in Formation 13. Once this constraint was introduced, optimized movement increased from 12.65 to 24.37 units, leaving only a 1.02% improvement over the actual choreography (24.62 units). This suggests that most of the apparent inefficiency in the unconstrained solution resulted from ignoring the choreographic requirement to preserve T as the focal member.

Repeat the same process for Formation change 14 → 15.

```
Add this
f1, f2 = 14, 15

dancers = list(formations[f1].keys())
target_names = list(formations[f2].keys())
targets = list(formations[f2].values())

cost_matrix = np.array([
    [distance(formations[f1][dancer], target) for target in targets]
```

```
    for dancer in dancers
])

# WEIGHTED TEST:
FOCUS PENALTY = 5
B → Y's slot
T → T's slot
A → B's slot
W → A's slot
Y → W's slot
Movement distance: 38.44
```

Interpretation: For Formation 14→15, a focus penalty of 5 was the minimum value required to preserve T's intended focal slot. This increased the optimized movement distance from 34.41 to 38.44 units. Compared with the actual choreography at 40.27 units, only a 4.54% improvement remained. As with Formation 12→13, part of the apparent geometric inefficiency can therefore be explained by the need to preserve the focal member.

Next, we tackle the more interesting Formation 15 rule: W should remain beside T. That cannot be represented exactly the same way as "protect T's slot," because it is a relationship between two dancers rather than one protected position.

4. Add W's "stay beside T" constraint
- Purpose: Formation 15 has a second artistic rule: T must keep T's focus slot, and W should remain beside T.

The choreography does not require W to occupy exactly ((2,-2)). What matters is that W remains sufficiently close to T and does not move into a position that obstructs T's visual focus. For example, a position such as ((0,-2)) could still satisfy the artistic intention even though it is not one of the original Formation 15 coordinates.

Therefore, protecting W's original slot - which is what I usually do - would unnecessarily restrict the optimizer and confuse two different ideas]:

- positional preservation: W must occupy one specific coordinate;
- relational preservation: W must maintain a particular spatial relationship with T. The second interpretation is more appropriate here.

Instead of fixing W to ((2,-2)), I define an acceptable region relative to T. With T fixed at [T=(0,-3),]. W must satisfy 2 conditions.

First, W must remain within a specific proximity radius r = 1 of T: $d(W,T) \leq r$, where the Euclidean distance between $W(x_W, y_W)$ and $T(x_T, y_T)$ is defined as:

$$d(W,T) = \sqrt{(x_W - x_T)^2 + (y_W - y_T)^2}$$

For the initial model, setting r = 1 permits positions such as (0, -2) relative to the origin, while excluding positions that place W beyond the allowable threshold.

Second, W must not be positioned in front of T to avoid obstructing the focal member. Under the coordinate system, this yields the inequality: $y_W \geq y_T$

The corresponding adjacency penalty function $P_{\text{adj}}(W, T)$ is formulated as:

$$P_{\text{adj}}(W,T) = \begin{cases} 0, & \text{if } d(W,T) \leq 1 \text{ and } y_W \geq y_T \\ \lambda_W, & \text{otherwise} \end{cases}$$

- Why allow new or slightly modified target coordinates?

The original Hungarian model assumes that the target formation consists of five fixed slots and that optimization only determines which dancer occupies each slot.

For this constraint, I deliberately relax that assumption. W may occupy a nearby coordinate that was not present in the original target formation, provided that the modified position still satisfies the artistic relationship with T.

I chose this approach because the original target coordinates represent one choreographer-designed realization of the formation rather than the only geometrically possible realization. If the objective of the project is to explore whether movement can be made more efficient without destroying the choreographic intention, forcing every dancer to occupy exactly the original five coordinates would be unnecessarily restrictive.

The model therefore begins to optimize two related decisions:

1. assignment: which dancer should occupy which role or region of the formation;
2. local positioning: whether a target coordinate can be adjusted slightly while preserving the intended visual structure.

This makes the model more flexible than the original Linear Assignment Problem, but also more realistic.

Evaluation: Allowing modified coordinates introduces an important trade-off. On one hand, it prevents the model from treating choreography as a set of completely immovable dots. Small spatial adjustments are realistic because formations can often tolerate slight differences in spacing while retaining the same visual effect. On the other hand, allowing unrestricted new coordinates would make optimization too easy. The algorithm could simply move target positions wherever convenient and achieve artificially low distances, rather than genuinely optimizing the choreography.

For this reason, any newly generated position should be treated as a local modification rather than a completely free coordinate. The acceptable region must remain constrained by choreographic rules such as proximity to T, direction relative to T, stage boundaries, and eventually minimum spacing from other dancers.

→ *"What is the most movement-efficient position W can occupy while still fulfilling W's choreographic role beside T?"*

Add this code block near the top after
    return weighted_matrix

_____

```
def add_focus_penalty(cost_matrix, dancers, target_names, focus_dancer, penalty):
    weighted_matrix = cost_matrix.copy()

    for i, dancer in enumerate(dancers):
        for j, target_name in enumerate(target_names):
            if target_name == focus_dancer and dancer != focus_dancer:
                weighted_matrix[i, j] += penalty

    return weighted_matrix


def generate_w_positions(t_position, radius=1, step=0.5):
    tx, ty = t_position
    positions = []

    x = tx - radius

    while x <= tx + radius:
        y = ty

        while y <= ty + radius:
            point = (round(x, 2), round(y, 2))

            if point != t_position and distance(point, t_position) <= radius:
                positions.append(point)

            y += step

        x += step

    return positions
```

Why here: both are reusable helper functions, so they belong together near the top before the main calculations.

Why this implementation: There are infinitely many points within one unit of T, so the computer needs a discrete search space. I will sample the stage every `0.5` coordinate units.

The loop only checks: $y_W >= y_T$ because it begins at T's y-coordinate and searches upward, so W can never be placed in front of T.

_____

```
t_position = formations[15]["T"]

w_candidates = generate_w_positions(

    t_position,

    radius=1,

    step=0.5

)

print("\nALLOWED W POSITIONS AROUND T:")

for position in w_candidates:

    print(position, "distance from T:", round(distance(position, t_position), 2))
```

MORE CORRECT VERSION SO THAT OUTPUT ~1.0

Why this is better: instead of filling the whole semicircle, we sample points along its boundary: d(W, T) = 1. And angles from 0o to 180o automatically guarantee yW >= yT.

```
def generate_w_positions(t_position, radius=1, angle_step=15):

    tx, ty = t_position

    positions = []

    for angle in range(0, 181, angle_step):

        radians = math.radians(angle)

        x = tx + radius * math.cos(radians)

        y = ty + radius * math.sin(radians)

        positions.append((round(x, 2), round(y, 2)))

    return positions
```

Also replace with

```
w_candidates = generate_w_positions(

    t_position,

    radius=1,

    angle_step=15
```

)

ALLOWED W POSITIONS AROUND T:
(-1, -3) distance from T: 1.0
(-0.5, -3) distance from T: 0.5
(-0.5, -2.5) distance from T: 0.71
(0.0, -2.5) distance from T: 0.5
(0.0, -2.0) distance from T: 1.0
(0.5, -3) distance from T: 0.5
(0.5, -2.5) distance from T: 0.71
(1.0, -3) distance from T: 1.0

_____
ALLOWED W POSITIONS AROUND T:
(1.0, -3.0) distance from T: 1.0
(0.97, -2.74) distance from T: 1.0
(0.87, -2.5) distance from T: 1.0
(0.71, -2.29) distance from T: 1.0
(0.5, -2.13) distance from T: 1.0
(0.26, -2.03) distance from T: 1.0
(0.0, -2.0) distance from T: 1.0
(-0.26, -2.03) distance from T: 1.0
(-0.5, -2.13) distance from T: 1.0
(-0.71, -2.29) distance from T: 1.0
(-0.87, -2.5) distance from T: 1.0
(-0.97, -2.74) distance from T: 1.0
(-1.0, -3.0) distance from T: 1.0

Now because this new coordinate represents W's role specifically, we must ensure W actually goes to that candidate position. Otherwise, the Hungarian algorithm might believe that this is a new cool position that A/Y/B can be sent there.

4.1 Add a function that creates each modified formation

- Purpose: This gives me a clean way to test all 13 possible W positions one by one without permanently changing your original Formation 15 data.
- After this, I'll calculate the movement cost for every candidate and find which W position gives the best constrained solution.

```
def make_modified_formation15(w_position):

    modified = formations[15].copy()

    modified["W"] = w_position

    return modified
```

```
# TEST ONE MODIFIED W POSITION
w_position = (0, -2)
modified_15 = make_modified_formation15(w_position)

# W is fixed to the new position
w_distance = distance(formations[14]["W"], w_position)

# Everyone except W is still assigned using Hungarian
remaining_dancers = ["B", "T", "A", "Y"]
remaining_targets = ["B", "T", "A", "Y"]
remaining_cost_matrix = np.array([
    [
        distance(formations[14][dancer], modified_15[target])
        for target in remaining_targets
    ]
    for dancer in remaining_dancers
])
weighted_remaining = add_focus_penalty(
    remaining_cost_matrix,
    remaining_dancers,
    remaining_targets,
    focus_dancer="T",
    penalty=5
)
rows, cols = linear_sum_assignment(weighted_remaining)
print("\nTEST W POSITION:", w_position)
print("W → modified W position", w_position, "distance:", round(w_distance, 2))
for r, c in zip(rows, cols):
    print(
        remaining_dancers[r],
        "→",
        remaining_targets[c] + "'s slot"
    )
total_distance = (
    w_distance
    + remaining_cost_matrix[rows, cols].sum()
)
print("Total movement distance:", round(total_distance, 2))
```

Why written this way: W is forced to the candidate position, while B/T/A/Y are still optimized. T's existing focus penalty of 5 remains active.

```
# TEST ALL ALLOWED W POSITIONS

results = []

for w_position in w_candidates:
```

```
modified_15 = make_modified_formation15(w_position)
# W is fixed to this candidate position
w_distance = distance(formations[14]["W"], w_position)
remaining_dancers = ["B", "T", "A", "Y"]
remaining_targets = ["B", "T", "A", "Y"]
remaining_cost_matrix = np.array([
    [
        distance(formations[14][dancer], modified_15[target])
        for target in remaining_targets
    ]
    for dancer in remaining_dancers
])
weighted_remaining = add_focus_penalty(
    remaining_cost_matrix,
    remaining_dancers,
    remaining_targets,
    focus_dancer="T",
    penalty=5
)
rows, cols = linear_sum_assignment(weighted_remaining)
total_distance = (
    w_distance
    + remaining_cost_matrix[rows, cols].sum()
)
results.append((w_position, total_distance))
```

```
    print(
        "W position:", w_position,
        "Total distance:", round(total_distance, 2)
    )
```

- Purpose: Find which valid position beside T gives the lowest total movement.
- Why I wrote the code how I wrote it =): the only variable changing is W's position. Everything else e.g. T's focus constraint and Hungarian optimization stays constant, so the comparison is fair.

Then add this block!

```
best_position, best_distance = min(
    results,
    key=lambda x: x[1]
)
print("\nBEST W POSITION:", best_position)
print("BEST TOTAL DISTANCE:", round(best_distance, 2))
```

- Purpose: whichever valid W coordinate has the smallest total is the preferred constrained Formation 15 position.

Output 1:
TEST W POSITION: (0, -2)
W → modified W position (0, -2) distance: 12.65
B → A's slot
T → T's slot
A → B's slot
Y → Y's slot
Total movement distance: 38.09
Output 2:
W position: (1.0, -3.0) Total distance: 39.37
W position: (0.97, -2.74) Total distance: 39.25
W position: (0.87, -2.5) Total distance: 39.08
W position: (0.71, -2.29) Total distance: 38.86
W position: (0.5, -2.13) Total distance: 38.61
W position: (0.26, -2.03) Total distance: 38.35
W position: (0.0, -2.0) Total distance: 38.09
W position: (-0.26, -2.03) Total distance: 37.86

```
W position: (-0.5, -2.13) Total distance: 37.66
W position: (-0.71, -2.29) Total distance: 37.52
W position: (-0.87, -2.5) Total distance: 37.45
W position: (-0.97, -2.74) Total distance: 37.45
W position: (-1.0, -3.0) Total distance: 37.53

BEST W POSITION: (-0.87, -2.5)
BEST TOTAL DISTANCE: 37.45
TINYYY change to check that part
    print(
        "W position:", w_position,
        "Total distance:", round(total_distance, 4)
    )
print("\nBEST W POSITION:", best_position)
print("BEST TOTAL DISTANCE:", round(best_distance, 4))

W position: (-0.87, -2.5) Total distance: 37.4494
W position: (-0.97, -2.74) Total distance: 37.4495

BEST W POSITION: (-0.87, -2.5)
BEST TOTAL DISTANCE: 37.4494
```

Interpretation: When W was allowed to occupy any valid position exactly one unit from T, the minimum constrained solution placed W at approximately (−0.87,−2.50). Total movement fell to 37.45 units, representing a 7.00% improvement over the actual choreography. This is more efficient than the T-focus-only fixed-slot solution (38.44 units), suggesting that preserving an artistic relationship does not necessarily require preserving an exact coordinate.

For 14→15:

- Actual choreography: 40.27
- Pure Hungarian: 34.41 → 14.57% improvement
- T-focus only: 38.44 → 4.54% improvement
- T-focus + flexible W adjacency: 37.4494 → 7.00% improvement

Before visualizing it, I need to know where B, T, A, Y, and W actually end up to see if the dancer arrangement makes sense choreographic-ally.

```
best_w_position = (-0.87, -2.5)
modified_15 = make_modified_formation15(best_w_position)

w_distance = distance(formations[14]["W"], best_w_position)

remaining_dancers = ["B", "T", "A", "Y"]
remaining_targets = ["B", "T", "A", "Y"]
remaining_cost_matrix = np.array([
    [
```

```
        distance(formations[14][dancer], modified_15[target])
        for target in remaining_targets
    ]
    for dancer in remaining_dancers
])
weighted_remaining = add_focus_penalty(
    remaining_cost_matrix,
    remaining_dancers,
    remaining_targets,
    focus_dancer="T",
    penalty=5
)
rows, cols = linear_sum_assignment(weighted_remaining)
print("\nBEST CONSTRAINED 14 → 15")
print("W →", best_w_position)
for r, c in zip(rows, cols):
    print(
        remaining_dancers[r],
        "→",
        remaining_targets[c] + "'s slot",
        modified_15[remaining_targets[c]]
    )
```

```
BEST CONSTRAINED 14 → 15
W → (-0.87, -2.5)
B → A's slot (-2, 1)
T → T's slot (0, -3)
A → B's slot (-8, 0)
Y → Y's slot (-4, -2)
```

Evaluation- Sequence-continuity problem

The constrained 14→15 solution assigns B to A's target slot and A to B's target slot. Although this reduces movement within the individual transition, the reassignment may be inefficient across the choreography as a whole. Formation 16 returns to dancer-specific roles, meaning B and A will have to cross or exchange positions again immediately afterward. This exposes a limitation of optimizing each formation transition independently: a locally optimal assignment at time t may increase movement or reduce visual coherence at time t+1.

Therefore, an assignment should not be evaluated solely by its immediate transition cost. The model must also consider continuity across consecutive formations, discouraging unnecessary role swaps that have to be reversed shortly afterward.

If B takes A's slot in Formation 15, then for 15→16:

B starts at (-2, 1)

A starts at (-8, 0).

So before deciding how large a "continuity penalty" should be, I need to carry the optimized 14→15 dancer positions forward into 15→16 and calculate what actually happens to check how "expensive" the B/A swap is.

Carry the optimized Formation 15 positions forward

- Purpose: Tell Python where each dancer actually ended after my constrained 14→15 solution.

```
carried_formation15 = {
    "B": (-2, 1),          # B took A's slot
    "T": (0, -3),          # T kept T's slot
    "A": (-8, 0),          # A took B's slot
    "W": (-0.87, -2.5),    # modified W position
    "Y": (-4, -2)          # Y kept Y's slot
}
```

→ The code: previously, Python still assumed Formation 15 was the original formation. This dictionary records the actual end state produced by our optimized transition.

THEN, Calculate what happens in 15 → 16

```
print("\nCARRIED-FORWARD 15 → 16")
carried_total = 0
for dancer in carried_formation15:
    start = carried_formation15[dancer]
    end = formations[16][dancer]
    d = distance(start, end)
    carried_total += d
    print(
        dancer,
        start, "→", end,
        "distance:", round(d, 2)
    )
print("15 → 16 distance after previous optimization:", round(carried_total, 2))
sequence_total = 37.4494 + carried_total
print("Combined 14 → 15 → 16 distance:", round(sequence_total, 2))
```

Interpretation:

Now B and A must travel from the positions they actually reached, rather than magically resetting to their original Formation 15 coordinates.

The comparison is: actual sequence=40.27+15.21=55.48 VS. Constrained optimized sequence = 37.4494+new downstream cost

```
CARRIED-FORWARD 15 → 16
B (-2, 1) → (-4, 0) distance: 2.24
T (0, -3) → (1, -3) distance: 1.0
```

```
A (-8, 0) → (4, 0) distance: 12.0
W (-0.87, -2.5) → (2, -2) distance: 2.91
Y (-4, -2) → (0, -1) distance: 4.12
15 → 16 distance after previous optimization: 22.27
Combined 14 → 15 → 16 distance: 59.72
```

Interpretation: Since the optimized two-transition total becomes larger than 55.48, the B/A swap is a perfect example of a local optimization that actually makes the overall sequence worse. This demonstrates that minimizing each transition independently can produce unnecessary temporary role swaps and increase movement later in the sequence. The optimization objective therefore needs to account for consecutive transitions rather than only the immediate formation change.

!! REDO For the 14→15→16 case, the objective is: $$C_{\text{sequence}} = D_{14 \to 15} + D_{15 \to 16}$$

subject to:

- T retains the focal role in Formation 15.
- W is exactly 1 unit from T.
- W does not block T.
- Everyone still occupies a unique position.
- The Formation 15 arrangement is allowed to change.
- We judge that arrangement by what it costs both to enter and to leave.

Instead of each assignment cost being only:

$$c_{ij} = d(\text{Formation } 14_i, \text{Formation } 15 \text{ slot}_j)$$

I now make it: $$c_{ij}^{\text{seq}} = d(14_i, 15_j) + d(15_j, 16_i)$$

So Python asks: "*If dancer ii takes this Formation 15 position, how much will they travel both getting there AND leaving it for Formation 16?*" That automatically punishes pointless B/A swaps without inventing an arbitrary continuity penalty.

```
# SEQUENCE-AWARE TEST: 14 → 15 → 16 FOR (0, -2)
w_position = (0, -2)
# W is fixed beside T
w_sequence_cost = (
    distance(formations[14]["W"], w_position)
    + distance(w_position, formations[16]["W"])
)
remaining_dancers = ["B", "T", "A", "Y"]
remaining_targets = ["B", "T", "A", "Y"]
sequence_cost_matrix = np.array([
    [
        distance(formations[14][dancer], formations[15][target])
        +
```

```
        distance(formations[15][target], formations[16][dancer])
        for target in remaining_targets
    ]
    for dancer in remaining_dancers
])

sequence_weighted = add_focus_penalty(
    sequence_cost_matrix,
    remaining_dancers,
    remaining_targets,
    focus_dancer="T",
    penalty=5
)
rows, cols = linear_sum_assignment(sequence_weighted)
print("\nSEQUENCE-AWARE TEST W =", w_position)
print("W →", w_position)
for r, c in zip(rows, cols):
    print(
        remaining_dancers[r],
        "→",
        remaining_targets[c] + "'s Formation 15 slot"
    )
sequence_total = (
    w_sequence_cost
    + sequence_cost_matrix[rows, cols].sum()
)
print("Total 14 → 15 → 16 distance:", round(sequence_total, 2))
```

→ Why I wrote how I wrote: Each possible Formation 15 assignment now includes its future consequence in Formation 16. I am optimizing the two-transition sequence, not just 14→15.

Output 1:

SEQUENCE-AWARE TEST W = (0, -2)
W → (0, -2)
B → B's Formation 15 slot
T → T's Formation 15 slot
A → A's Formation 15 slot
Y → Y's Formation 15 slot
Total 14 → 15 → 16 distance: 55.57
Interpretation: Test position W=(0,-2): $$55.57$$ compared with the actual sequence: $$55.48$$ So it is only 0.09 units worse. More importantly, I have only tested one of W's 13 valid positions.

```
sequence_results = []
for w_position in w_candidates:
    w_sequence_cost = (
        distance(formations[14]["W"], w_position)
        + distance(w_position, formations[16]["W"])
    )
```

```
    remaining_dancers = ["B", "T", "A", "Y"]
    remaining_targets = ["B", "T", "A", "Y"]
    sequence_cost_matrix = np.array([
        [
            distance(formations[14][dancer], formations[15][target])
            +
            distance(formations[15][target], formations[16][dancer])
            for target in remaining_targets
        ]
        for dancer in remaining_dancers
    ])
    sequence_weighted = add_focus_penalty(
        sequence_cost_matrix,
        remaining_dancers,
        remaining_targets,
        focus_dancer="T",
        penalty=5
    )
    rows, cols = linear_sum_assignment(sequence_weighted)
    sequence_total = (
        w_sequence_cost
        + sequence_cost_matrix[rows, cols].sum()
    )
    sequence_results.append((w_position, sequence_total))
    print(
        "W position:", w_position,
        "Sequence total:", round(sequence_total, 4)
    )
```

ALSO ADD

```
best_w, best_sequence = min(
    sequence_results,
    key=lambda x: x[1]
)

print("\nBEST SEQUENCE-AWARE W POSITION:", best_w)
print("BEST SEQUENCE TOTAL:", round(best_sequence, 4))
```

Output 2:

```
CARRIED-FORWARD 15 → 16
B (-2, 1) → (-4, 0) distance: 2.24
T (0, -3) → (1, -3) distance: 1.0
A (-8, 0) → (4, 0) distance: 12.0
W (-0.87, -2.5) → (2, -2) distance: 2.91
Y (-4, -2) → (0, -1) distance: 4.12
15 → 16 distance after previous optimization: 22.27
Combined 14 → 15 → 16 distance: 59.72
W position: (1.0, -3.0) Sequence total: 56.2632
W position: (0.97, -2.74) Sequence total: 55.9979
W position: (0.87, -2.5) Sequence total: 55.7904
```

```
W position: (0.71, -2.29) Sequence total: 55.6573
W position: (0.5, -2.13) Sequence total: 55.5909
W position: (0.26, -2.03) Sequence total: 55.5663
W position: (0.0, -2.0) Sequence total: 55.5698
W position: (-0.26, -2.03) Sequence total: 55.5933
W position: (-0.5, -2.13) Sequence total: 55.6431
W position: (-0.71, -2.29) Sequence total: 55.7237
W position: (-0.87, -2.5) Sequence total: 55.8392
W position: (-0.97, -2.74) Sequence total: 55.9868
W position: (-1.0, -3.0) Sequence total: 56.166

BEST SEQUENCE-AWARE W POSITION: (0.26, -2.03)
BEST SEQUENCE TOTAL: 55.5663
```

Interpretation: Very unfortunately, the best sequence-aware constrained solution is only 0.0863 units worse than the real choreography. I should not conclude yet that the original is better, because our W search is still coarse: I only sampled every 15°. → angle_step=1

[would be stupid for me to copy like 180 lines of output so]

BEST SEQUENCE-AWARE W POSITION: (0.16, -2.01)

BEST SEQUENCE TOTAL: 55.5648

Evaluation: Why does the constrained sequence not outperform the original choreography?

Reason 1: The $$W$$ constraint may currently be stricter than the choreography itself

The most important issue is that I defined $$W$$'s acceptable position as being exactly one Euclidean unit from T.

However, in the original Formation 15: $$T = (0, -3)$$ and $$W = (2, -2)$$

Their actual Euclidean separation is: $$d(T, W) = \sqrt{(2 - 0)^2 + (-2 + 3)^2} = \sqrt{5} \approx 2.24$$

Therefore, the original choreography itself does not satisfy the constraint $$d(W, T) = 1$$.

This means the comparison is not currently completely equivalent. The optimized model is being asked to satisfy an additional spatial restriction that the original choreography does not satisfy, yet it is still being compared against the original choreography's distance.

The fact that the constrained solution comes within only $$0.15\%$$ of the original despite this additional restriction may therefore indicate that the model is already performing quite efficiently.

More importantly, this suggests that the adjacency constraint should probably not be defined as an arbitrary Euclidean distance of exactly one coordinate unit. A better definition would need to reflect what "beside T" means within the scale of the actual formation

Reason 2: The original choreography is already highly sequence-efficient
The earlier single-transition optimization initially suggested that Formation 14 to 15 contained considerable inefficiency: $$40.27 \rightarrow 34.41$$ or a $$14.57\%$$ reduction.
However, this saving depended largely on temporarily exchanging dancer roles, particularly A and B. Once the resulting Formation 15 positions were carried forward into Formation 16, the next transition increased substantially: $$15.21 \rightarrow 22.27$$
The combined locally optimized sequence became: $$59.72$$
which is worse than the original: $$55.48$$
When the objective function was changed to consider both transitions simultaneously,
$$c_{ij}^{\text{seq}} = d(F14_i, F15_j) + d(F15_j, F16_i)$$
the Hungarian algorithm independently returned B, T, A, Y to their own intended Formation 15 roles. This suggests that what initially appeared to be inefficient movement in Formation 14 to 15 may actually serve a longer-term purpose. The choreographer appears to accept slightly greater immediate travel in order to place dancers advantageously for the following formation.

Therefore, the original choreography is not simply optimized for the shortest individual transition. It already contains a form of temporal planning across formations.
Reason 3: The optimized search space is still more restricted than real choreography

Although W''s coordinate is allowed to change, the other Formation 15 target positions remain fixed. The model therefore searches:

- different dancer assignments;
- different legal positions for W;

but it does not allow small adjustments to B, A, Y or other formation geometry. A choreographer has a much larger continuous design space. Multiple dancers may shift slightly while preserving the same recognizable formation. Thus, the current optimization is not finding the mathematical optimum over every possible choreography. It is finding the optimum within a deliberately restricted family of formations.

Collision-feasibility result: After fixing T to the focal slot in Formation 13, I exhaustively checked all $4! = 24$ remaining dancer-slot assignments. None produced a completely crossing-free set of straight-line movement vectors. Therefore, increasing the collision penalty could not eliminate the T×Y crossing; the limitation came from the feasible formation geometry rather than insufficient weighting.

Collision comparison: Unlike Formation 12→13, Formation 14→15 does contain a crossing-free assignment while preserving T's focal role. The minimum crossing-free solution has a total distance of 38.44 units, identical to the focus-preserving optimum. Therefore, collision avoidance introduces no additional distance cost for this transition. This shows that collision feasibility depends strongly on the geometry of each formation rather than on a universal penalty value.

FULL ROUTINE COLLISION SCREENING

Transition | Hungarian | H-Cross | Focus Distance | F-Cross | Crossing-Free? | CF Distance

11 → 12 | 42.14 | 0 | 42.20 | 1 | No | -

12 → 13 | 12.65 | 0 | 24.37 | 1 | No | -

13 → 14 | 9.22 | 0 | 9.22 | 0 | Yes | 9.22

14 → 15 | 34.41 | 0 | 38.44 | 0 | Yes | 38.44

15 → 16 | 15.21 | 0 | 15.21 | 0 | Yes | 15.21

- Pure Hungarian finds zero-crossing solutions for all five transitions.
- Once I enforce T's focal role, crossings appear in 11→12 and 12→13.
- For those two transitions, no crossing-free focus-preserving assignment exists at all within the fixed target slots.
- For 13→14, 14→15, 15→16, the focus-preserving optimum is already crossing-free, so collision avoidance costs nothing extra.

FULL ROUTINE SUMMARY

Transition | Original | Hungarian | H-Cross | Focus | F-Cross | Crossing-Free? | CF Distance
11 → 12 | 42.20 | 42.14 | 0 | 42.20 | 1 | No | -
12 → 13 | 24.62 | 12.65 | 0 | 24.37 | 1 | No | -
13 → 14 | 9.22 | 9.22 | 0 | 9.22 | 0 | Yes | 9.22
14 → 15 | 40.27 | 34.41 | 0 | 38.44 | 0 | Yes | 38.44
15 → 16 | 15.21 | 15.21 | 0 | 15.21 | 0 | Yes | 15.21

Step 36 onward — Generalization Test

The first version of the project was developed and tested on one five-member OnHit choreography. Although the solver functions are not hard-coded to five dancers, testing only one routine would not demonstrate that the program can actually be reused on new choreography. The purpose of this stage is therefore to test whether a new routine can be analyzed simply by replacing the formation data, without changing the optimization functions themselves.

Step 36 — Select a second choreography

Identify approximately 3 consecutive formations, giving two formation transitions to analyze. The formations should preferably have clearly visible dancer positions and limited camera obstruction so that approximate stage coordinates can be assigned consistently.

The second routine does not have to contain five dancers. In fact, using a routine with a different number of performers would provide a stronger generalization test because the program obtains the performer names directly from the formation dictionary rather than assuming a fixed group size.

For example:

SECOND_FORMATIONS = { 1: { "A": (x, y), "B": (x, y), ... }, 2: { ... }, 3: { ... }}

The same performers should appear in each formation within the selected sequence.

Why: The original project used five performers and formations 11–16. A second dataset tests whether the solver represents a general dancer-to-slot assignment problem rather than a program written specifically around those original coordinates.

Step 37 — Encode the new formations as input data

Estimate each performer's position using the same two-dimensional stage-coordinate system used for the first routine. The exact coordinate scale is less important than applying it consistently across all three formations.

If a formation has an identifiable focal or center member, record that separately:

SECOND_FOCUS = { 2: "A", 3: "A"}

If preserving a particular performer is not important for one formation:

SECOND_FOCUS = { 2: None, 3: None}

Only the input data should change at this stage. Functions such as `solve_transition()`, `solve_collision_aware()`, `find_best_crossing_free()`, and `analyze_routine()` should remain unchanged.

Why: This is the actual test of software generalization. If a new choreography requires rewriting the solver, then the program is still routine-specific. If only the formation coordinates and artistic constraints need to be replaced, the optimization logic is genuinely reusable.

Step 38 — Run the existing analysis pipeline unchanged

Pass the second routine directly into the existing routine-level function:

second_results = analyze_routine(SECOND_FORMATIONS, SECOND_FOCUS)print("\nSECOND ROUTINE")print_routine_summary(second_results)

For every transition, the program should again calculate:

1. the movement distance in the original choreography;
2. the minimum-distance Hungarian assignment;
3. the number of straight-line path crossings;
4. the best assignment under the focal-member constraint; and
5. whether a completely crossing-free, focus-preserving assignment exists.

Why: Running exactly the same analysis on unseen formation data provides a simple reproducibility check. The goal is not necessarily to show that the algorithm improves every transition, but to show that the same framework can evaluate different choreography without manual changes to the optimization logic.

Step 39 — Evaluate whether the generalization test succeeds

The generalization test will be considered successful if the new routine can be processed by changing only:

formation coordinates

focal-member information

and no changes are required to the solver functions.

The numerical result does not need to show an improvement in every transition. A new routine may produce a large reduction in movement, almost no reduction, unavoidable crossings, or a trade-off between focal-member preservation and distance. These are all legitimate outputs of the model.

The important result is that the same program can diagnose those cases consistently.

One limitation should also be recorded: the Hungarian minimum-distance solver scales efficiently to considerably larger groups, but the collision-aware component currently checks every possible assignment. Its search space therefore grows factorially,

$n!$

so the exhaustive collision analysis is intended primarily for relatively small groups. For example, five dancers require only $5! = 120$ assignments, while seven dancers require $7! = 5,040$.

Intended outcome

After this stage, the project can support a more defensible claim such as:

> Built a reusable Python framework for analyzing group-choreography transitions using minimum-cost assignment, focal-member constraints, and movement-path collision screening, and tested the same solver on multiple choreography sequences.

That is stronger than simply showing that the original five-member example worked, while still being appropriately modest for the scope of the project.

Dancer	Formation 45	Formation 46	Formation 47	Formation 48	Formation 49	Formation 50
B	(-10, 2)	(-6, 4)	(-6, 2)	(-6, 1)	(-6, -1)	(-6, -1)
T	(-10, 0)	(3, 4)	(3, 2)	(3, -1)	(3, 2)	(3, -1)
A	(0, 6)	(0, 4)	(0, 4)	(0, 1)	(0, -1)	(0, -1)
W	(5, -2)	(2, -1)	(-3, 0)	(-3, -1)	(-3, 2)	(-3, -1)
Y	(4, -1)	(4, -2)	(6, 4)	(6, 1)	(6, -1)	(6, -1)

Additional relational constraint (same as the previous sequence): In the second sequence, W and Y are choreographically required to remain spatially close. Their separation is constrained to at most 2 units in Formation 45 and 3 units in Formation 46. The original coordinates satisfy these limits √2 and √5, respectively). Because the tested sequence begins at Formation 45, the Formation 45 condition is treated as validation of the initial state, while the Formation 46 condition is enforced during optimization of the 45→46 transition.

...	# Relationally constrained solution
	constrained_4546 = solve_collision_aware(
	SECOND_FORMATIONS[45],
	SECOND_FORMATIONS[46],
	
	pair_constraints=SECOND_PAIR_CONSTRAINTS[46],
	)

	print("\nWith W-Y distance <= 3:") for item in constrained_4546["assignments"]: print(item["dancer"], "→", item["target_slot"] + "'s slot") print("Distance:", round(constrained_4546["distance"], 2)) print("Crossings:", constrained_4546["crossings"])
SECOND SEQUENCE TEST: 45 → 46 Without W-Y constraint: B → B's slot T → A's slot A → T's slot W → Y's slot Y → W's slot Distance: 21.85	With W-Y distance <= 3: B → B's slot T → A's slot A → T's slot W → Y's slot Y → W's slot Distance: 21.85 Crossings: []

YAY! The second sequence introduced a new relational constraint, but the unconstrained optimum already satisfied it, so enforcing the constraint produced no additional movement cost.

SECOND SEQUENCE: FORMATIONS 45 → 50

FULL ROUTINE SUMMARY

Transition | Original | Hungarian | H-Cross | Focus | F-Cross | Crossing-Free? | CF Distance

45 → 46 | 24.24 | 21.85 | 0 | 21.85 | 0 | Yes | 21.85

46 → 47 | 15.42 | 14.22 | 0 | 14.22 | 0 | Yes | 14.22

47 → 48 | 11.00 | 11.00 | 0 | 11.00 | 0 | Yes | 11.00

48 → 49 | 12.00 | 12.00 | 0 | 12.00 | 0 | Yes | 12.00

49 → 50 | 6.00 | 6.00 | 0 | 6.00 | 0 | Yes | 6.00

Generalization test 01: I applied the same optimization framework to a second, previously unused sequence of six formations (45–50), producing five new transitions. No changes were made to the core assignment, collision-detection, or exhaustive-search logic. The new sequence also introduced a relational choreography constraint requiring W and Y to remain within 3 coordinate units in Formation 46. The unconstrained minimum-distance assignment already satisfied this condition, so enforcing the constraint did not increase movement cost. The solver reduced total movement by approximately 9.9% for 45→46 and 7.8% for 46→47, while the remaining three transitions were already distance-optimal. All five optimized transitions were crossing-free under the straight-line path model.

These results suggest that the framework can be reused on previously unseen formation sequences by changing only the coordinate data and choreography-specific constraints. They also show that optimization does not necessarily improve every transition: several later formation changes were already minimum-distance assignments.